

LANGCHAIN

MASTERCLASS

From Beginner to Industrial-Grade AI Engineering
LangChain · LangGraph · LCEL · LangSmith · RAG · Agents · Production

Beginner → Expert	16 Levels	100+ Exercises	Industrial Capstone	Production-Ready
-------------------	-----------	----------------	---------------------	------------------

Course Overview

This masterclass is a university-level, FAANG-quality curriculum for engineers who want deep, production-grade fluency in the LangChain ecosystem. You will move from Python fundamentals through LLM engineering, advanced RAG, multi-agent systems, observability, and full production deployment — finishing with an industrial capstone that mirrors real startup and enterprise systems.

Who Is This For?

- Senior software engineers new to LangChain and LLM application development
- Python developers with solid fundamentals wanting production AI engineering skills
- Backend engineers building AI-powered APIs, chatbots, RAG systems, or agents
- Anyone aiming for AI engineering roles at top-tier companies

What You Will Build

Project	Description
♦ Chatbot with Memory	Stateful conversational AI with persistent context
♦ PDF RAG System	Multi-document retrieval-augmented generation
♦ SQL Agent	Natural-language-to-SQL with tool use
♦ Multi-Tool Agent	Autonomous agent with web search and code exec
♦ Streaming API	Real-time token streaming via FastAPI
♦ Evaluation Framework	LangSmith-powered automated quality evaluation
♦ Multi-Agent Platform	Supervisor + specialist agent orchestration
♦ Enterprise Capstone	Full production AI knowledge platform with Docker + CI/CD

Full Learning Roadmap

Level	Topic	Duration
L0	Python & Software Foundations	4–6 wks
L1	LLM Fundamentals	1–2 wks
L2	LangChain Foundations	1–2 wks
L3	LCEL and Chains	1 wk
L4	Prompt Engineering Deep Dive	1–2 wks
L5	Memory Systems	1 wk
L6	RAG Systems	2–3 wks
L7	Agents and Tool Calling	2 wks
L8	LangGraph and Stateful Systems	2–3 wks
L9	Multi-Agent Architectures	1–2 wks
L10	Observability and Evaluation	1 wk
L11	Production AI Engineering	2 wks
L12	Advanced Optimization	1 wk
L13	Security and Guardrails	1 wk
L14	Distributed AI Systems	1–2 wks
L15	Industrial Capstone	3–4 wks

Estimated Total Timeline: 26–40 weeks (self-paced)

Accelerated pace (full-time study): 3–4 months. Part-time (10–15 hrs/week): 7–10 months. The capstone alone is designed to take 3–4 weeks of focused work.

Table of Contents

■ Prerequisites

- Python Standard Libraries (40+ modules)
- Advanced Python Patterns
- Third-Party Libraries & Ecosystem
- Software Engineering Principles

■ Level 0 — Python & Software Foundations

- 0.1 Core Python Mastery
- 0.2 Async/Await & Concurrency
- 0.3 Decorators, Generators, Context Managers
- 0.4 OOP, SOLID, Design Patterns
- 0.5 Testing with pytest
- 0.6 Clean Architecture

■ Level 1 — LLM Fundamentals

- 1.1 Transformer Architecture
- 1.2 Tokenization & Context Windows
- 1.3 Completion vs Chat APIs
- 1.4 Temperature, Top-P, Sampling
- 1.5 OpenAI SDK Deep Dive
- 1.6 Cost Optimization Basics

■ Level 2 — LangChain Foundations

- 2.1 LangChain Architecture Overview
- 2.2 Models & Integrations
- 2.3 Prompts & Templates
- 2.4 Output Parsers
- 2.5 Document Loaders & Text Splitters
- 2.6 Embeddings

■ Level 3 — LCEL and Chains

- 3.1 LCEL Philosophy & Runnable Protocol
- 3.2 Pipe Operator & Chain Composition
- 3.3 Parallel Execution
- 3.4 Streaming in LCEL
- 3.5 Fallbacks & Error Handling
- 3.6 Custom Runnables

■ Level 4 — Prompt Engineering Deep Dive

- 4.1 Few-Shot & Zero-Shot Prompting
- 4.2 Chain-of-Thought Reasoning
- 4.3 Structured Output Prompting
- 4.4 Prompt Templates & Variables
- 4.5 Prompt Injection Defense
- 4.6 Dynamic Prompt Construction

■ Level 5 — Memory Systems

- 5.1 Short-Term vs Long-Term Memory
- 5.2 ConversationBufferMemory
- 5.3 ConversationSummaryMemory
- 5.4 Vector Store Memory
- 5.5 External Memory (Redis, DB)
- 5.6 Memory Evaluation

■ Level 6 — RAG Systems

- 6.1 RAG Architecture Overview
- 6.2 Document Ingestion Pipeline
- 6.3 Chunking Strategies
- 6.4 Vector Stores (Chroma, FAISS, Qdrant)
- 6.5 Retrieval Strategies
- 6.6 Advanced RAG (HyDE, Re-ranking, Fusion)
- 6.7 RAG Evaluation

■ Level 7 — Agents and Tool Calling

- 7.1 Agent Architecture
- 7.2 Tool Definition & Registration
- 7.3 ReAct Agent Pattern
- 7.4 OpenAI Function Calling
- 7.5 Structured Outputs
- 7.6 Custom Tools
- 7.7 Tool Error Recovery

■ Level 8 — LangGraph and Stateful Systems

- 8.1 LangGraph Philosophy
- 8.2 StateGraph & State Management
- 8.3 Nodes, Edges & Conditional Routing
- 8.4 Checkpointing & Persistence
- 8.5 Human-in-the-Loop
- 8.6 Streaming in LangGraph

■ Level 9 — Multi-Agent Architectures

- 9.1 Multi-Agent Design Patterns
- 9.2 Supervisor Architecture
- 9.3 Hierarchical Agent Systems
- 9.4 Agent Communication Protocols
- 9.5 Parallelism in Agent Graphs
- 9.6 MCP Concepts

■ Level 10 — Observability and Evaluation

- 10.1 LangSmith Setup & Tracing
- 10.2 Custom Evaluators
- 10.3 LLM-as-Judge
- 10.4 Dataset & Benchmark Creation
- 10.5 Monitoring in Production

■ Level 11 — Production AI Engineering

- 11.1 FastAPI + LangChain Integration
- 11.2 Async Streaming Endpoints
- 11.3 Authentication & Rate Limiting
- 11.4 Caching Strategies
- 11.5 Retry & Fallback Patterns
- 11.6 Docker & Deployment

■ Level 12 — Advanced Optimization

- 12.1 Token Budget Management
- 12.2 Batch Processing
- 12.3 Model Selection Strategy
- 12.4 Latency Optimization
- 12.5 Cost Monitoring & Alerting

■ Level 13 — Security and Guardrails

- 13.1 Prompt Injection Prevention
- 13.2 Output Validation
- 13.3 Secrets Management
- 13.4 PII Detection & Redaction
- 13.5 Abuse Prevention
- 13.6 Compliance Considerations

■ Level 14 — Distributed AI Systems

- 14.1 Celery Task Queues
- 14.2 Horizontal Scaling Patterns
- 14.3 Load Balancing LLM Calls
- 14.4 Database-Backed State
- 14.5 Event-Driven AI Workflows

■ Level 15 — Industrial Capstone

- 15.1 System Architecture Design
- 15.2 Implementation Roadmap
- 15.3 Full Stack Integration
- 15.4 Testing Strategy
- 15.5 Deployment & CI/CD
- 15.6 Monitoring & Alerting
- 15.7 Cost & Performance Optimization
- 15.8 Security Hardening

Python Standard Library Mastery

Before writing a single line of LangChain code, you must be fluent in the standard library. LangChain internals, FastAPI backends, and production AI systems all rely heavily on these modules. Skipping this foundation creates invisible gaps that will cost you days of debugging later.

os / sys / pathlib

Purpose: File system and process interaction

- `os.environ` — read environment variables (API keys, config)
- `os.path` / `pathlib.Path` — cross-platform path handling
- `sys.argv` — CLI argument parsing without `argparse`
- `sys.exit()` — controlled process termination
- `pathlib.Path.glob()` — recursive file discovery for document loaders

```
# python - os / sys / pathlib
p = Path('data') / 'docs' / 'file.txt' # path composition
env_key = os.environ.get('OPENAI_API_KEY', '') # safe read
```

json

Purpose: Serialization for LLM inputs/outputs

- `json.loads()` / `json.dumps()` — core serialization
- `json.load()` / `json.dump()` — file I/O
- `indent=` parameter for human-readable output
- `default=` hook for custom types (e.g. `datetime`, `UUID`)

```
# python - json
data = json.loads(llm_response) # parse structured output
json.dump(trace, f, indent=2, default=str) # safe serialization
```

re

Purpose: Regex for output parsing and validation

- `re.search()` / `re.findall()` — pattern extraction
- `re.sub()` — sanitisation and PII redaction
- Named groups (`?P...`) — structured extraction
- `re.compile()` — pre-compile patterns for performance

```
# python - re
```

```
pattern = re.compile(r"```json (.*) ```", re.DOTALL)
match = pattern.search(llm_output); data = json.loads(match.group(1))
```

asyncio

Purpose: Async event loop — core to modern LangChain

- `asyncio.run()` — entry point for async programs
- `async def` / `await` — coroutine definition and suspension
- `asyncio.gather()` — concurrent coroutine execution
- `asyncio.Queue` — async-safe producer/consumer queues
- `asyncio.Semaphore` — concurrency limiting for API rate limits

```
# nathan - asyncio

async def fetch_all(queries: list[str]):
    tasks = [acall_llm(q) for q in queries]
    return await asyncio.gather(*tasks)
```

logging

Purpose: Structured observability for AI systems

- `logging.getLogger(__name__)` — module-scoped logger
- Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Handlers: StreamHandler, FileHandler, RotatingFileHandler
- Formatters: JSON formatters for log aggregation (ELK, CloudWatch)
- `structlog` library builds on top for structured logging

```
# nathan - logging

logger = logging.getLogger(__name__)
logger.info("LLM call", extra={"tokens": 412, "model": "gpt-4o"})
```

dataclasses / typing

Purpose: Type-safe data modelling

- `@dataclass` — lightweight data containers
- `field(default_factory=list)` — mutable defaults
- `typing.Optional`, `Union`, `Literal` — type annotations
- `typing.TypedDict` — typed dicts for LangChain state
- `typing.Protocol` — structural subtyping

```
# nathan - dataclasses / typing

from dataclasses import dataclass, field
@dataclass
class AgentState:
    messages: list = field(default_factory=list)
    iteration: int = 0
```

collections

Purpose: Efficient data structures

- `defaultdict` — auto-initialising dicts for grouping

- Counter — frequency counting for token analysis
- deque — O(1) append/pop for sliding window memory
- OrderedDict — ordered caching patterns
- namedtuple — lightweight immutable records

```
# python - collections
from collections import deque, Counter
window = deque(maxlen=10) # sliding context window
token_freq = Counter(tokens) # most_common(10)
```

functools / itertools

Purpose: Functional programming utilities

- functools.lru_cache / cache — memoisation for embeddings
- functools.partial — partial application for config injection
- functools.reduce — fold operations on chains
- itertools.chain — flatten document chunks
- itertools.batched (3.12) — batch API calls

```
# python - functools / itertools
from functools import lru_cache
@lru_cache(maxsize=1024)
def embed(text: str) -> list[float]: ...
# Cache expensive embedding calls automatically
```

contextlib

Purpose: Context managers for resource safety

- @contextmanager — create context managers from generators
- contextlib.suppress — silent exception handling
- contextlib.asynccontextmanager — async resource management
- ExitStack — dynamic context manager composition

```
# python - contextlib
from contextlib import asynccontextmanager
@asynccontextmanager
async def lifespan(app): # FastAPI lifespan pattern
    await db.connect()
    yield
    await db.disconnect()
```

concurrent.futures

Purpose: Thread/process pools for blocking I/O

- ThreadPoolExecutor — parallel blocking I/O (PDF loading)
- ProcessPoolExecutor — CPU-bound tasks (embeddings batch)
- as_completed() — stream results as they arrive
- submit() vs map() — individual vs bulk submission

```
# python - concurrent.futures
```

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(load_pdf, p) for p in paths]
    docs = [f.result() for f in futures]
```

Additional Standard Library Modules

The following modules are equally important and must be studied:

Module	Key Usage in AI Engineering
<code>threading</code>	Low-level thread management, Lock, Event, Condition variables
<code>multiprocessing</code>	Process-based parallelism, Pipe, Queue, Manager, Pool
<code>datetime</code> / <code>time</code>	Timestamps for tracing, TTL caches, rate-limit windows
<code>sqlite3</code>	Lightweight local DB for conversation history and checkpoints
<code>traceback</code>	Exception formatting for structured error logging
<code>inspect</code>	Runtime introspection — used by LangChain internally for tool schemas
<code>abc</code>	Abstract base classes for defining agent and tool interfaces
<code>enum</code>	Enumerated constants for model names, response formats, states
<code>csv</code> / <code>io</code>	CSV ingestion for document loaders and evaluation datasets
<code>pickle</code>	Object serialisation for checkpointing agent state
<code>hashlib</code> / <code>secrets</code> / <code>uuid</code>	Content hashing, token generation, unique IDs for traces
<code>tempfile</code> / <code>shutil</code>	Temporary file handling for document processing pipelines
<code>subprocess</code>	Shell command execution from agents (code execution tool)
<code>queue</code>	Thread-safe queues for producer/consumer streaming patterns
<code>heapq</code>	Priority queues for scheduling and ranked retrieval
<code>weakref</code>	Weak references to avoid memory leaks in caches
<code>socket</code> / <code>http</code> / <code>urllib</code>	Low-level networking, HTTP utilities, URL parsing
<code>statistics</code> / <code>math</code> / <code>decimal</code>	Numerical utilities for score aggregation and evaluation
<code>signal</code>	Process signal handling for graceful shutdown in production

Advanced Python Patterns

Decorators

Functions that wrap other functions to extend behaviour without modifying source code. Used extensively in FastAPI (route registration), LangChain (tool decoration), and caching layers.

- `@functools.wraps(func)` — preserve metadata
- `@lru_cache` — memoization
- `@retry (tenacity)` — automatic retries
- `@app.get('/')` — FastAPI route
- `@tool` — LangChain tool registration

```
# python - Decorators
def retry(max_attempts=3):
    def decorator(func):
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try: return await func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts-1: raise
                    await asyncio.sleep(2 ** attempt)
            return wrapper
        return decorator
```

Generators & Iterators

Generators produce values lazily, enabling memory-efficient streaming of tokens, chunks, and large document sets. LangChain's streaming interface is built on Python generators and async generators.

- `yield` — suspend and produce a value
- `yield from` — delegate to sub-generator
- `async for` — iterate async generators
- `__iter__ / __next__` — iterator protocol
- `itertools.islice` — limit generator output

```
# python - Generators & Iterators
async def stream_tokens(prompt: str):
    async for chunk in llm.astream(prompt):
        yield chunk.content # async generator

# Consume in FastAPI SSE endpoint
async def sse_handler():
    async for token in stream_tokens(prompt):
        yield f'data: {token}\n\n'
```

Context Managers

Guarantee resource cleanup with `__enter__`/`__exit__` or via `@contextmanager`. Critical for database connections, file handles, and tracing spans in production AI systems.

- `__enter__` / `__exit__` — protocol methods
- `@contextmanager` — generator-based context managers
- `@asynccontextmanager` — async version
- `contextlib.ExitStack` — dynamic composition
- Used in: DB sessions, HTTP clients, LangSmith traces

```
# python - Context Managers
@asynccontextmanager
async def llm_trace(run_name: str):
    span = tracer.start_span(run_name)
    try:
        yield span
    except Exception as e:
        span.set_error(str(e))
        raise
    finally:
        span.end()
```

Async/Await & Event Loops

Python's async I/O model allows concurrent LLM API calls without threads. LangChain exposes async interfaces (`ainvoke`, `astream`, `abatch`) for all components. Understanding the event loop is essential for production performance.

- `asyncio.run()` — start the event loop
- `asyncio.gather()` — fan-out concurrent calls
- `asyncio.Semaphore` — throttle API concurrency
- `asyncio.Queue` — decoupled producer/consumer
- `loop.run_in_executor()` — run blocking code

```
# python - Async/Await & Event Loops
sem = asyncio.Semaphore(10) # max 10 concurrent LLM calls

async def rate_limited_call(query: str):
    async with sem:
        return await llm.ainvoke(query)

results = await asyncio.gather(*[
    rate_limited_call(q) for q in queries
])
```

Descriptors & Metaclasses

Descriptors control attribute access (`__get__`/`__set__`/`__delete__`). Metaclasses customise class creation. Both appear in Pydantic (v1) and SQLAlchemy internals. Understanding them helps debug framework issues.

- `__get__ / __set__` — descriptor protocol
- `@property` — Python's built-in descriptor
- `type.__new__` — metaclass hook
- `ABCMeta` — abstract base class metaclass
- Pydantic uses `__init_subclass__` for field registration

```
# Python — Descriptors & Metaclasses

class ValidatedField:
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, obj, objtype=None):
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, str):
            raise TypeError
        obj.__dict__[self.name] = value
```

OOP, SOLID Principles & Design Patterns

Production LangChain systems require solid OOP. These principles appear directly in LangChain's own codebase and in every well-engineered AI backend.

Principle	Name	AI Engineering Application
S	Single Responsibility	One class, one reason to change. DocumentLoader loads. Splitter splits. Don't merge.
O	Open/Closed	Open for extension, closed for modification. Extend via subclassing or composition.
L	Liskov Substitution	Subtypes must be substitutable for base types. All retrievers are interchangeable.
I	Interface Segregation	Many small interfaces > one fat interface. Separate Retriever from Ranker.
D	Dependency Inversion	Depend on abstractions. Inject LLM, VectorStore, Memory — never instantiate inside.

Third-Party Library Ecosystem

These libraries form the scaffolding of every production AI engineering system. Master them before writing production LangChain code.

pydantic

Data validation and settings management

The backbone of LangChain's type system. Every LangChain component uses Pydantic models. Understanding Pydantic deeply lets you write custom components and debug schema errors.

- BaseModel — define typed, validated data models
- Field(...) — defaults, validation, aliases
- model_validator / field_validator — custom validation
- model_json_schema() — generate JSON Schema for tool calls
- Settings (pydantic-settings) — environment variable injection

```
# nathan — pydantic
from pydantic import BaseModel, Field, model_validator
class QueryRequest(BaseModel):
    query: str = Field(..., min_length=1, max_length=2048)
    top_k: int = Field(5, ge=1, le=20)
    @model_validator(mode='after')
    def sanitize(self): self.query = self.query.strip(); return self
```

fastapi

High-performance async API framework

LangChain backends are almost universally deployed as FastAPI applications. FastAPI's async support, automatic OpenAPI docs, and dependency injection make it ideal for AI endpoints.

- @app.post / @app.get — route registration
- Depends() — dependency injection (auth, DB sessions)
- BackgroundTasks — fire-and-forget post-response tasks
- StreamingResponse — SSE token streaming
- lifespan — startup/shutdown resource management

```
# nathan — fastapi
from fastapi import FastAPI, Depends
from fastapi.responses import StreamingResponse
app = FastAPI()
@app.post('/chat')
async def chat(req: QueryRequest):
    async def gen():
        async for token in chain.astream(req.query):
            yield f'data: {token}\n\n'
    return StreamingResponse(gen(), media_type='text/event-stream')
```

httpx

Modern async HTTP client

Preferred over requests for async LangChain systems. Used for calling external APIs, webhooks, and custom LLM providers.

- `httpx.AsyncClient` — async HTTP client
- `client.get / post` — HTTP verbs
- `timeout=` — request timeout configuration
- `limits=` — connection pool limits
- `event_hooks` — request/response middleware

```
# python - httpx

async with httpx.AsyncClient(timeout=30.0) as client:
    response = await client.post(
        'https://api.openai.com/v1/chat/completions',
        headers={'Authorization': f'Bearer {key}'},
        json={'model': 'gpt-4o', 'messages': msgs}
    )
    return response.json()
```

sqlalchemy + alembic

ORM and database migrations

Used to persist conversation history, user sessions, agent checkpoints, and evaluation results. Alembic handles schema migrations.

- `AsyncSession` — async database sessions
- `select()` — type-safe query building
- `relationship()` — ORM associations
- `alembic revision --autogenerate` — schema migration
- `alembic upgrade head` — apply migrations

```
# python - sqlalchemy + alembic

from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
async def get_history(user_id: str, session: AsyncSession):
    result = await session.execute(
        select(Message).where(Message.user_id == user_id)
        .order_by(Message.created_at)
    )
    return result.scalars().all()
```

tenacity

Production retry logic

LLM APIs fail. Rate limits, timeouts, and transient errors are facts of production life. Tenacity provides declarative retry policies with exponential backoff.

- `@retry` — decorator-based retry

- `stop_after_attempt(n)` — max retries
- `wait_exponential(multiplier, min, max)` — backoff
- `retry_if_exception_type()` — selective retry
- `before_sleep=` — logging between attempts

```
# python - tenacity

from tenacity import retry, stop_after_attempt, wait_exponential
@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=60),
    retry=retry_if_exception_type(RateLimitError),
)
async def resilient_llm_call(prompt: str):
    return await llm.ainvoke(prompt)
```

redis + celery

Caching and background tasks

Redis caches embeddings and LLM responses. Celery handles async document ingestion, batch evaluation, and long-running agent tasks outside the request cycle.

- `redis.asyncio` — async Redis client
- `await client.get/set/setex` — key-value operations
- `@celery.task` — define background tasks
- `task.delay()` — enqueue task
- Redis as Celery broker + result backend

```
# python - redis + celery

import redis.asyncio as redis
r = redis.Redis.from_url(settings.REDIS_URL)
async def cached_embed(text: str) -> list[float]:
    key = f'embed:{hashlib.md5(text.encode()).hexdigest()}'
    cached = await r.get(key)
    if cached: return json.loads(cached)
    vec = await embedder.aembed_query(text)
    await r.setex(key, 86400, json.dumps(vec)) # 24h TTL
    return vec
```

pytest + pytest-asyncio

Testing framework

Every production AI system needs automated tests. pytest's fixture system, parametrize, and mocking integrate with LangSmith datasets for evaluation.

- `@pytest.fixture` — reusable test setup
- `@pytest.mark.asyncio` — async test support
- `@pytest.mark.parametrize` — data-driven tests
- `unittest.mock.AsyncMock` — mock async LLM calls
- `pytest-cov` — coverage reporting


```
# python -m pytest + pytest-asyncio

import pytest
from unittest.mock import AsyncMock
@pytest.mark.asyncio
async def test_rag_returns_relevant_docs(mock_retriever):
    mock_retriever.aget_relevant_documents = AsyncMock(
        return_value=[Document(page_content='Paris is the capital')]
    )
    result = await chain.ainvoke('What is the capital of France?')
    assert 'Paris' in result['answer']
```

LLM Fundamentals

Before building with LangChain, you must understand what Large Language Models are, how they work at a conceptual and practical level, and how to interact with them via APIs. This knowledge prevents magical thinking and enables principled debugging.

1.1 Transformer Architecture (Conceptual)

LLMs are transformer neural networks trained on massive text corpora to predict the next token. The key mechanism is self-attention, which allows the model to weigh relationships between all tokens in the context window simultaneously.

Concept	Explanation
Tokens	Subword units — roughly 1 token $\approx$ 0.75 words in English. tiktoken encodes text to tokens.
Context Window	Maximum tokens the model can process at once (e.g. 128K for GPT-4o). Exceeding this causes truncation.
Self-Attention	Each token attends to all other tokens — enables long-range dependency capture.
Temperature	Controls randomness. 0.0 = deterministic; 1.0+ = creative/random. Use low for factual tasks.
Top-P (Nucleus)	Cumulative probability threshold for token sampling. 0.9 = top 90% probability mass.
Max Tokens	Hard limit on output length. Prevents runaway generation and controls cost.
System Prompt	Invisible instruction preamble that shapes model behaviour throughout the conversation.
Stop Sequences	Strings that terminate generation early — useful for structured output extraction.

1.2 OpenAI SDK Deep Dive

```
# python - OpenAI SDK: async chat completion
from openai import AsyncOpenAI
import tiktoken

client = AsyncOpenAI() # reads OPENAI_API_KEY from env

async def chat(messages: list[dict], model='gpt-4o') -> str:
    response = await client.chat.completions.create(
        model=model,
        messages=messages,
```

```

        temperature=0.0,
        max_tokens=1024,
    )
    return response.choices[0].message.content

# Token counting before sending
enc = tiktoken.encoding_for_model('gpt-4o')
def count_tokens(messages: list[dict]) -> int:
    return sum(len(enc.encode(m['content'])) for m in messages)

```

1.3 Streaming Completions

```

# python — streaming with async generator

async def stream_chat(prompt: str):
    stream = await client.chat.completions.create(
        model='gpt-4o',
        messages=[{'role': 'user', 'content': prompt}],
        stream=True,
    )
    async for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            yield delta # token by token

# Usage
async for token in stream_chat('Explain RAG in 3 sentences'):
    print(token, end='', flush=True)

```

Common Mistakes

- Ignoring token counts — exceeding context causes silent truncation, not an error
- Temperature=1.0 for factual tasks — causes hallucination and inconsistency
- Not handling rate limits — always wrap LLM calls in retry logic (tenacity)
- Synchronous SDK in async context — use AsyncOpenAI, not OpenAI, in async code
- Hardcoding API keys — always use environment variables or secret managers

Interview Questions — Level 1

- What is a token and how does tokenization affect cost?
- Explain the difference between temperature and top-p sampling.
- Why does the system prompt persist across a conversation?
- What happens when you exceed the context window limit?
- How would you implement rate limit handling for LLM APIs?

LangChain Foundations

LangChain is a framework for building LLM applications. It provides abstractions over models, prompts, memory, retrievers, and agents — enabling composable, testable, and production-ready AI pipelines.

2.1 Architecture Overview

Package	Purpose
<code>langchain-core</code>	Base abstractions: Runnable, BaseLLM, BaseMemory, Document. No vendor dependencies.
<code>langchain</code>	High-level chains, agents, and utilities built on core.
<code>langchain-community</code>	Third-party integrations (vector stores, document loaders, LLMs).
<code>langchain-openai</code>	OpenAI-specific integration with ChatOpenAI, OpenAIEmbeddings.
<code>langgraph</code>	Stateful, cyclic agent graphs. The recommended agent framework.
<code>langsmith</code>	Observability, tracing, evaluation, and dataset management.

2.2 ChatModels and LLMs

```
# python - model setup with environment variables

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

llm = ChatOpenAI(
    model='gpt-4o',
    temperature=0.0,
    max_tokens=1024,
    timeout=30,
    max_retries=3, # built-in retry
)

# Simple invocation
response = await llm.ainvoke([
    SystemMessage(content='You are a helpful assistant.'),
    HumanMessage(content='What is LangChain?'),
])
print(response.content)
```

2.3 Prompt Templates

```
# python — ChatPromptTemplate

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}. Be concise.'),
    ('human', '{question}'),
])

# Format and inspect
messages = prompt.format_messages(
    domain='Python',
    question='Explain async/await in 2 sentences.',
)

# Compose with model
chain = prompt | llm
result = await chain.ainvoke({'domain': 'Python', 'question': '...'})
```

2.4 Output Parsers

```
# python — Structured output with Pydantic

from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel

class ExtractedInfo(BaseModel):
    name: str
    date: str
    summary: str

parser = PydanticOutputParser(pydantic_object=ExtractedInfo)

prompt = ChatPromptTemplate.from_messages([
    ('system', 'Extract info from the text. {format_instructions}'),
    ('human', '{text}'),
]).partial(format_instructions=parser.get_format_instructions())

chain = prompt | llm | parser
info: ExtractedInfo = await chain.ainvoke({'text': raw_text})
```

2.5 Document Loaders & Text Splitters

```
# python — loading and splitting documents

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Load
loader = PyPDFLoader('report.pdf')
pages = await loader.aload()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
```

```
        chunk_overlap=200,  
        separators=['\n\n', '\n', '. ', ' ', ''],  
    )  
    chunks = splitter.split_documents(pages)  
    print(f'Split {len(pages)} pages into {len(chunks)} chunks')
```

LCEL — LangChain Expression Language

LCEL is LangChain's declarative chain composition syntax. It uses Python's pipe operator (`|`) to compose Runnable objects. Every component — prompts, models, parsers, retrievers — implements the Runnable interface and is therefore composable, streamable, and batchable.

3.1 The Runnable Protocol

- `invoke(input)` — synchronous call
- `ainvoke(input)` — async call
- `stream(input)` — sync streaming
- `astream(input)` — async streaming
- `batch(inputs)` — parallel batch
- `abatch(inputs)` — async batch

```
# python - LCEL chain composition

from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# Basic chain: prompt | model | parser
chain = prompt | llm | StrOutputParser()

# Parallel execution
parallel_chain = RunnableParallel(
    answer=chain,
    raw_docs=retriever,
)

# Pass-through with augmentation
rag_chain = (
    RunnablePassthrough.assign(context=retriever)
    | prompt
    | llm
    | StrOutputParser()
)

# Streaming
async for token in chain.astream({'question': 'What is RAG?'}):
    print(token, end='', flush=True)
```

3.2 Fallbacks

```
# python - chain with fallback
```

```

primary = ChatOpenAI(model='gpt-4o')
fallback = ChatOpenAI(model='gpt-4o-mini')

resilient_llm = primary.with_fallbacks([fallback])

# With retry
robust_llm = primary.with_retry(
    stop_after_attempt=3,
    wait_exponential_jitter=True,
).with_fallbacks([fallback])

```

3.3 Custom Runnables

```

# python - RunnableLambda for custom steps
from langchain_core.runnables import RunnableLambda

def format_docs(docs: list) -> str:
    return '\n\n'.join(d.page_content for d in docs)

# Wrap as Runnable
format_step = RunnableLambda(format_docs)

# Use in chain
rag_chain = retriever | format_step | prompt | llm | StrOutputParser()

```


RAG Systems — Retrieval-Augmented Generation

RAG is the most important architecture pattern in production LLM applications. Instead of relying solely on parametric knowledge (what the model learned during training), RAG dynamically retrieves relevant context from a knowledge base at inference time.

6.1 RAG Architecture

INDEXING PHASE: Load → Split → Embed → Store in Vector DB

RETRIEVAL PHASE: Embed query → Similarity search → Retrieve top-k chunks

GENERATION PHASE: Inject chunks into prompt → LLM generates grounded answer

6.2 Vector Store Setup

```
# python - chroma vector store

from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

embeddings = OpenAIEmbeddings(model='text-embedding-3-large')

# Build from documents
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db',
    collection_name='company_docs',
)

# Retriever interface
retriever = vectorstore.as_retriever(
    search_type='mmr', # Maximum Marginal Relevance
    search_kwargs={'k': 6, 'fetch_k': 20, 'lambda_mult': 0.5},
)
```

6.3 Complete RAG Chain

```
# python - production RAG chain with sources

from langchain_core.runnables import RunnableParallel, RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
```

```
def format_docs(docs): return '\n\n'.join(d.page_content for d in docs)

prompt = ChatPromptTemplate.from_messages([
    ('system', '''Answer using ONLY the context below.
Context: {context}
If the answer is not in context, say I don't know.'''),
    ('human', '{question}'),
])

rag_chain = RunnableParallel(
    context=retriever | format_docs,
    question=RunnablePassthrough(),
) | prompt | llm | StrOutputParser()

answer = await rag_chain.ainvoke('What is our refund policy?')
```

6.4 Advanced RAG Techniques

Technique	Description
HyDE	Hypothetical Document Embedding — generate a hypothetical answer, embed it, then retrieve. Improves semantic match.
Re-ranking	Use a cross-encoder (Cohere Rerank, BGE) to re-score retrieved docs. Improves precision at cost of latency.
RAG-Fusion	Generate multiple query variants, retrieve for each, fuse results with Reciprocal Rank Fusion.
Parent-Child Chunks	Embed small chunks for precision, return larger parent chunks for context richness.
Self-RAG	Model decides when to retrieve, evaluates its own answer, and re-retrieves if needed.
RAPTOR	Recursive tree of summaries for hierarchical retrieval across large corpora.

LangGraph — Stateful Agent Systems

LangGraph is the recommended framework for building stateful, multi-step agent workflows. Unlike simple chains, LangGraph supports cycles, conditional branching, persistence, streaming, and human-in-the-loop patterns — making it suitable for complex autonomous agents.

8.1 Core Concepts

StateGraph — Graph where each node can read and write to a typed state object

State — TypedDict or Pydantic model defining the data flowing through the graph

Nodes — Python functions or Runnables that transform state

Edges — Connections between nodes — static or conditional

Checkpoint — Persistence layer — enables resume, branching, and HITL

Streaming — `astream_events()` for real-time token + node output

8.2 ReAct Agent with LangGraph

```
# python — minimal LangGraph ReAct agent

from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage
import operator

class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], operator.add]

def should_continue(state: AgentState) -> str:
    last = state['messages'][-1]
    return 'tools' if last.tool_calls else END

def call_model(state: AgentState):
    response = llm_with_tools.invoke(state['messages'])
    return {'messages': [response]}

graph = StateGraph(AgentState)
graph.add_node('agent', call_model)
graph.add_node('tools', ToolNode(tools))
graph.set_entry_point('agent')
graph.add_conditional_edges('agent', should_continue)
graph.add_edge('tools', 'agent')
```

```

app = graph.compile(checkpointer=MemorySaver())
result = await app.ainvoke(
    {'messages': [HumanMessage('Search for recent AI papers')]},
    config={'configurable': {'thread_id': 'user-123'}},
)

```

8.3 Persistence & Human-in-the-Loop

```

# python - interrupt for human approval

from langgraph.checkpoint.postgres import PostgresSaver

# Compile with DB-backed persistence
checkpointer = PostgresSaver.from_conn_string(settings.DATABASE_URL)
app = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=['tools'], # pause before tool execution
)

# Run until interrupt
thread = {'configurable': {'thread_id': 'order-456'}}
state = await app.ainvoke(input, config=thread)

# Human reviews and approves
await app.ainvoke(None, config=thread) # resume

```

Production AI Engineering

Building a working LangChain prototype is 10% of the work. The remaining 90% is making it reliable, scalable, observable, and secure in production. This level covers everything between 'it works on my laptop' and 'it serves 10,000 users per day.'

11.1 FastAPI + LangChain Backend Architecture

```
# python - production FastAPI AI backend

from fastapi import FastAPI, Depends, HTTPException
from fastapi.responses import StreamingResponse
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    await db.connect()
    await redis_client.ping()
    app.state.chain = build_rag_chain()
    yield
    # Shutdown
    await db.disconnect()

app = FastAPI(lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=['*'])

@app.post('/api/v1/chat')
async def chat_endpoint(
    req: ChatRequest,
    user=Depends(verify_jwt),
    rate_limiter=Depends(check_rate_limit),
):
    async def generate():
        async for token in app.state.chain.astream(req.dict()):
            yield f'data: {json.dumps({"token": token})}\n\n'
        yield 'data: [DONE]\n\n'
    return StreamingResponse(generate(), media_type='text/event-stream')
```

11.2 Caching Strategy

```
# python - multi-level caching
```

```

from langchain.globals import set_llm_cache
from langchain_community.cache import RedisSemanticCache

# Semantic cache: cache similar questions together
set_llm_cache(RedisSemanticCache(
    redis_url=settings.REDIS_URL,
    embedding=OpenAIEmbeddings(),
    score_threshold=0.95, # similarity threshold
))

# Embedding cache
store = RedisStore(redis_url=settings.REDIS_URL)
embedder = CacheBackedEmbeddings.from_bytes_store(
    underlying_embeddings=OpenAIEmbeddings(),
    document_embedding_cache=store,
    namespace='embed-v1',
)

```

11.3 Docker Deployment

```

# Dockerfile — multi-stage production build

FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12 /usr/local/lib/python3.12
COPY . .

RUN adduser --disabled-password appuser && chown -R appuser /app
USER appuser

EXPOSE 8000
CMD ['uvicorn', 'main:app', '--host', '0.0.0.0', '--port', '8000',
    '--workers', '4', '--loop', 'uvloop']

```

11.4 Production Checklist

- Environment variables via .env + pydantic-settings (never hardcoded)
- Structured JSON logging with correlation IDs for every request
- Health check endpoints (/health, /ready) for load balancers
- Graceful shutdown with lifespan context manager
- Connection pooling for database and Redis
- Rate limiting per user/IP with Redis token buckets
- Request/response validation with Pydantic
- Error handling with structured error responses (no stack traces to clients)
- LangSmith tracing for every LLM call in production
- Circuit breaker pattern for downstream API failures
- Token budget enforcement to prevent cost overruns

- Input sanitisation to prevent prompt injection

Enterprise AI Knowledge Platform

The capstone is a fully production-grade, multi-tenant AI knowledge platform. It combines every major concept from the course into a system that could realistically be deployed at a startup or enterprise.

System Description

A multi-tenant document intelligence platform where organisations can upload their knowledge bases (PDFs, URLs, Notion, Confluence), and employees can query them via a chat interface powered by RAG and autonomous agents. The system includes real-time streaming, full observability, evaluation pipelines, and cost controls.

Required Technologies

Core Stack	Infrastructure
• LangGraph — agent orchestration	• Docker + docker-compose
• LangChain RAG — document retrieval	• GitHub Actions — CI/CD
• LangSmith — tracing and evaluation	• pytest + pytest-asyncio
• FastAPI — async REST + SSE	• JWT authentication
• Pydantic — validation	• Prometheus + Grafana
• PostgreSQL + pgvector — hybrid search	• OpenAI GPT-4o + embeddings
• Redis — caching + rate limiting	• Qdrant — vector store
• Celery — background ingestion	• Alembic — DB migrations

Project Structure

```
# directory structure
enterprise-ai-platform/
├── backend/
│   ├── app/
│   │   ├── api/           # FastAPI routers
│   │   └── agents/        # LangGraph agent definitions
```



```

■ ■ ■■ rag/           # RAG pipeline components
■ ■ ■■ memory/        # Memory and state management
■ ■ ■■ evaluation/     # LangSmith evaluators
■ ■ ■■ ingestion/      # Celery tasks for doc processing
■ ■ ■■ models/         # SQLAlchemy ORM models
■ ■ ■■ schemas/        # Pydantic schemas
■ ■ ■■ core/           # Config, security, dependencies
■ ■ ■■ main.py         # FastAPI application
■ ■■ tests/           # pytest test suite
■ ■■ alembic/         # Database migrations
■ ■■ Dockerfile
■ ■■ requirements.txt
■■■ frontend/         # React / Next.js chat UI
■■■ docker-compose.yml
■■■ .github/workflows/ # CI/CD pipelines
■■■ README.md

```

Implementation Phases

Phase 1 (Week 1)	Core infrastructure: FastAPI skeleton, DB setup, auth, health checks, Dockerfile
Phase 2 (Week 2)	RAG pipeline: document ingestion, chunking, embedding, vector store, basic Q&A;
Phase 3 (Week 3)	Agent layer: LangGraph multi-tool agent, tool definitions, streaming, HITL
Phase 4 (Week 4)	Production hardening: caching, rate limiting, LangSmith tracing, evaluation, CI/CD

Mandatory Requirements

- Multi-tenant: users only see their organisation's documents
- Streaming: all responses stream token-by-token via SSE
- Async throughout: no synchronous blocking code in FastAPI handlers
- Persistence: conversation history stored in PostgreSQL
- Evaluation: automated LangSmith evaluation suite with 20+ test cases
- Observability: every LLM call traced with metadata, tokens, latency
- Cost controls: per-user token budget with hard limits enforced
- Retry logic: tenacity-based retry on all LLM and vector DB calls
- Security: JWT auth, rate limiting, prompt injection prevention, input validation
- Testing: 80%+ code coverage, integration tests with mocked LLM calls
- Docker: entire stack runnable with docker-compose up
- CI/CD: GitHub Actions — lint, test, build, push on every PR

Bonus Challenges

- Add voice input/output (Whisper + TTS)

- Implement RAPTOR hierarchical indexing for large corpora
- Build a Notion/Confluence connector via their APIs
- Add A/B testing for different RAG configurations
- Deploy to AWS ECS or GCP Cloud Run with auto-scaling
- Implement a feedback loop: thumbs up/down → fine-tuning dataset
- Add multi-modal support (images in documents)
- Build a cost analytics dashboard with Grafana

Readiness Criteria

You are ready to call yourself an industrial-grade LangChain engineer when you can:

- Build and deploy a production RAG system from scratch in < 1 day
- Debug a LangGraph agent cycle using LangSmith traces without guessing
- Explain the tradeoffs between chunking strategies for a given use case
- Design a multi-agent system with clear state boundaries and error recovery
- Write a custom LangChain component that integrates with any third-party API
- Estimate and control LLM API costs for a given traffic profile
- Set up observability, alerting, and evaluation in under 2 hours
- Pass a senior AI engineering technical interview with confidence

Official Sources & References

Resource	URL
LangChain Documentation	https://python.langchain.com/docs/
LangGraph Documentation	https://langchain-ai.github.io/langgraph/
LangSmith Documentation	https://docs.smith.langchain.com/
OpenAI API Reference	https://platform.openai.com/docs/
FastAPI Documentation	https://fastapi.tiangolo.com/
Pydantic Documentation	https://docs.pydantic.dev/
Python Standard Library	https://docs.python.org/3/library/
Chroma Documentation	https://docs.trychroma.com/
Qdrant Documentation	https://qdrant.tech/documentation/
SQLAlchemy Documentation	https://docs.sqlalchemy.org/
Redis Documentation	https://redis.io/docs/

Celery Documentation	https://docs.celeryq.dev/
pytest Documentation	https://docs.pytest.org/
tiktoken Repository	https://github.com/openai/tiktoken
tenacity Documentation	https://tenacity.readthedocs.io/